Personal and Professional Project

Study, Design, Implementation and Commissioning of an Autonomous Robot for Unstructured Environments based on Embedded Artificial Intelligence

Eurobot 2026 Competition – *Winter is Coming*



Presented by:

Bensaid Mohamed Amine
Hmadi Malek
Jihed Sellini
Khalsi Maram

Supervisor: M. Abderrazak CHATTI

Defense date: 02/06/2026

Academic year 2025–2026

Acknowledgements

We would like to thank our supervisor, M. Abderrazak CHATTI, for his guidance and advice throughout this project. We also thank the members of the Aerobotix INSAT club for their technical and logistical support, as well as all those who contributed to the success of this project and the competition.

Contents

Acknowledgements **i**

Glossary and Abbreviations **v**

Presentation of the Institution – INSAT **1**

Problem Statement **2**

Technical Specifications **3**

General Introduction **4**

1 State of the Art and International Analysis **5**
1.1 Chapter Introduction 5
1.2 Eurobot 2026 Context – Winter is Coming 5
1.3 Main Actions and Scoring System 8
1.4 Panorama of Existing Solutions 8
1.5 Critical Analysis and Positioning 8
1.6 Chapter Conclusion 9

2 Design and Techno-Economic Study **10**
2.1 Chapter Introduction 10
2.2 Mechanical Design 10
2.2.1 Mechanical Design Methodology 11
2.2.2 Main Robot Mechanical Design 14
2.3 Electronic Design 18

2.4 Software Design . . . 21
2.4.1 ROS2 Architecture . . . 21
2.4.2 Match Strategy and Task Scheduling . . . 23
2.4.3 Navigation and Localization . . . 23
2.4.4 PID Motor Control (STM32) . . . 23
2.4.5 Odometry and Pose Correction . . . 24
2.4.6 Navigation and Trajectory Generation . . . 24
2.4.7 LIDAR Perception and Obstacle Management . . . 25
2.4.8 Path Planning and Occupancy Grid . . . 25
2.4.9 Vision and Detection . . . 27
2.4.10 Embedded AI for Pickability Prediction . . . 27
2.5 Techno-Economic Study . . . 28
2.6 Chapter Conclusion . . . 29

3 Simulation Before Implementation 30
3.1 Chapter Introduction . . . 30
3.2 Simulation Environment . . . 30
3.3 Navigation and Path Planning Simulation . . . 31
3.4 Control Loop Simulation . . . 32
3.5 Chapter Conclusion . . . 32

4 Implementation, Testing and Validation 33
4.1 Chapter Introduction . . . 33
4.2 Physical Implementation . . . 33
4.3 Software Deployment . . . 35
4.4 Unit Tests . . . 35
4.5 Integration Tests and Competition . . . 36
4.6 Observed Limitations . . . 36
4.7 Chapter Conclusion . . . 39

General Conclusion and Perspectives 40

Abstract 41

Bibliography and Webography 42

List of Figures

1.1 Official game area (global view). . . . 5
1.2 Eurobot 2026 playground mat (top view). . . . 6
1.3 Hazelnut crates and game elements. . . . 7
1.4 * . . . 7
1.5 * . . . 7

1.6 Nest and pantry (granary associated with the game area). 7

2.1 Concept A: articulated front collector (initial idea). 11
2.2 Concept B: two-stage lifting and reorientation (prototype views). 12
2.3 Concept C: rotating hinge with disc-based orientation (prototype views). . 12
2.4 Concept D: grasping and pneumatic subsystems (implementation views). . 13
2.5 Isometric CAD view of the robot. 16
2.6 Top CAD view of the robot. 17
2.7 Grasping system (gripper/suction cup). 17
2.8 Mounted pneumatic system. 18
2.9 PCB1/2 schematic (ESP32 + TB6612FNG drivers). 19
2.10 3D view of PCB1 and PCB2. 19
2.11 PCB3 schematic (MOSFETs for pumps/solenoid valves). 20
2.12 3D view of PCB3. 20
2.13 Photo of soldered PCBs (front side). 20
2.14 Block diagram of the ROS2 / micro-ROS architecture. 21
2.15 ROS2 node graph (rqt_graph). 22
2.16 Velocity PID control loop (inner loop). 24
2.17 Position PID control loop (outer loop). 24
2.18 Hybrid A* path planning block diagram. 26
2.19 Occupancy Grid (RViz). 26
2.20 AI pickability pipeline (feature extraction, normalization, inference, labeling). 28

3.1 RViz2: robot on the Eurobot 2026 map. 30
3.2 Planned trajectory (Hybrid A* on map). 31
3.3 Navigation PID curves (step response). 32

4.1 Assembled robot (front and side views). 33
4.2 Internal wiring. 34
4.3 PCBs photo. 34
4.4 ROS2 terminal screenshot (active topics, latency). 35
4.5 Competition photos – Tunisia (national qualifications). 36
4.6 Competition photos – France (international final). 37
4.7 Match score table. 38

List of Tables

1 Aerobotix INSAT competition record. 1
2 Technical specifications summary. 3

1.1 Eurobot 2026 scoring system summary. 8
1.2 Positioning of our choices against common alternatives. 9

2.1 ROS2 nodes and their main interfaces. 22
2.2 PID gains used for navigation. 23
2.3 Bill of Materials (BOM) – complete component list. 29

4.1 Main unit test results. 35

Glossary and Abbreviations

ROS2	Robot Operating System 2
micro-ROS	ROS2 extension for microcontrollers
PID	Proportional-Integral-Derivative controller
LIDAR	Light Detection And Ranging
MCU	Microcontroller Unit
STM32	ARM Cortex-M microcontroller
ESP32	Wi-Fi/Bluetooth microcontroller
ArUco	Visual markers for vision and localization
BOM	Bill of Materials
PCB	Printed Circuit Board
CAD	Computer-Aided Design
SBC	Single Board Computer
PAMI	Small Autonomous Mobile Robot (Petit Automate Mobile Indépendant)
EMA	Exponential Moving Average
PWM	Pulse Width Modulation

Presentation of the Institution – INSAT

The National Institute of Applied Sciences and Technology (INSAT) provides excellence-level training in electrical engineering, industrial computing, and embedded systems. The Aerobotix INSAT club drives the institute's participation in Eurobot, with a recognized culture of innovation and competitiveness at both national and international levels.

INSAT relies on a project-based pedagogy and hands-on coursework oriented towards Industry 4.0, cyber-physical systems, and autonomous robotics. This approach bridges theory and real-world engineering constraints: real-time systems, multi-domain integration, hardware robustness, and experimental validation. This PPP project fits directly within this professionalization framework.

The Aerobotix INSAT club structures the technical supervision and organizational aspects of the Eurobot participation. Teams develop standardized working methods (configuration management, incremental testing, documentation) and a performance culture built on continuous validation. This collective experience contributes to project quality and knowledge transfer between student cohorts.

Year	National Ranking	International Ranking
2022	1st	5th
2023	1st	–
2024	2nd	17th
2025	1st	19th
2026	1st	2nd worldwide (France)

Table 1: Aerobotix INSAT competition record.

Problem Statement

This project addresses several technical and scientific challenges that guided our design decisions:

- Precise autonomous navigation in an unstructured and dynamic environment.
- Real-time coordination between heterogeneous computing units (RPi5, STM32, ESP32).
- Robust localization without GPS, despite odometric drift and vibrations.
- Reactive obstacle avoidance using a LIDAR combined with a dynamic strategy.
- Reliable manipulation of hazelnut crates under match time constraints.

These constraints guided the development of the technical specifications and the overall robot architecture.

From a methodological standpoint, the project requires balancing sometimes contradictory requirements: performance versus cost, reactivity versus stability, software complexity versus maintainability, and robust execution over a 100-second match. The objective is not simply to complete a trajectory, but to guarantee reliable execution of a sequence of actions under real-world disturbances (moving opponent, wheel slip, sensor noise, load variations).

Technical Specifications

Requirement	Technical Specification
Central computing	Raspberry Pi 5, Ubuntu 24.04, ROS2 Jazzy
Real-time navigation	STM32F446RE, PID position/velocity loops
Actuators	3x ESP32,motors,encoders,sensors and pneumatic system
Communication	micro-ROS between RPi5 and MCUs
Obstacle detection	LIDAR LD19, 360°, frequency $\geq$ 10 Hz
Path planning	Hybrid A* in Python, dynamic replanning
Vision	Raspberry Pi Camera + OpenCV + ArUco markers
Autonomy	100-second match, 0 human intervention

Table 2: Technical specifications summary.

The specifications serve as the reference for all technical choices and validation tests. They are structured around critical functions (navigation, grasping, communication) and competition constraints. Each requirement is broken down into verifiable sub-requirements (precision, latency, robustness) to ensure progressive and objective validation.

General Introduction

Autonomous robotics is rapidly evolving towards more intelligent systems capable of perceiving, planning, and acting in complex environments. The Eurobot 2026 competition, themed *Winter is Coming*, imposes realistic constraints on perception, navigation, and physical interaction.

In this context, designing an autonomous robot is no longer a simple matter of assembling components, but a problem of global system integration. It requires a coherent hardware architecture, modular software, and algorithms that are robust in the face of uncertainty. This PPP emphasizes a systems engineering approach: every design choice is evaluated based on its impact on the robot's final behavior during a match.

The project has two objectives. First, to build a robot capable of performing game actions with precision and reliability. Second, to establish an evolutionary, reusable architecture for future editions. This dual ambition drives the design towards a distributed and modular solution built on ROS2 and micro-ROS.

This report is organized into four chapters: the first surveys the state of the art and provides an international analysis; the second develops the technical design and cost study; the third presents the simulation work; and the fourth describes the physical implementation, testing, and competition validation.

1 State of the Art and International Analysis

1.1 Chapter Introduction

This chapter presents the Eurobot 2026 context and a panorama of existing solutions in order to clearly position our approach.

1.2 Eurobot 2026 Context – Winter is Coming

The game table measures 3000 mm × 2000 mm and includes a starting nest, an elevated granary, and storage zones (pantries). Game elements include hazelnut crates, empty crates, a thermometer with a cursor, and PAMIs (small autonomous robots). Each match lasts 100 seconds.

Game actions require alternating between precise movements and manipulation phases. The robot must locate targets, align itself, perform a reliable grasp, then reach a drop zone without collision. During a match, the opponent constitutes a dynamic obstacle that forces real-time trajectory adaptation. Strategy is therefore just as decisive as mechanics or electronics.



Figure 1.1: Official game area (global view).



Figure 1.2: Eurobot 2026 playground mat (top view).

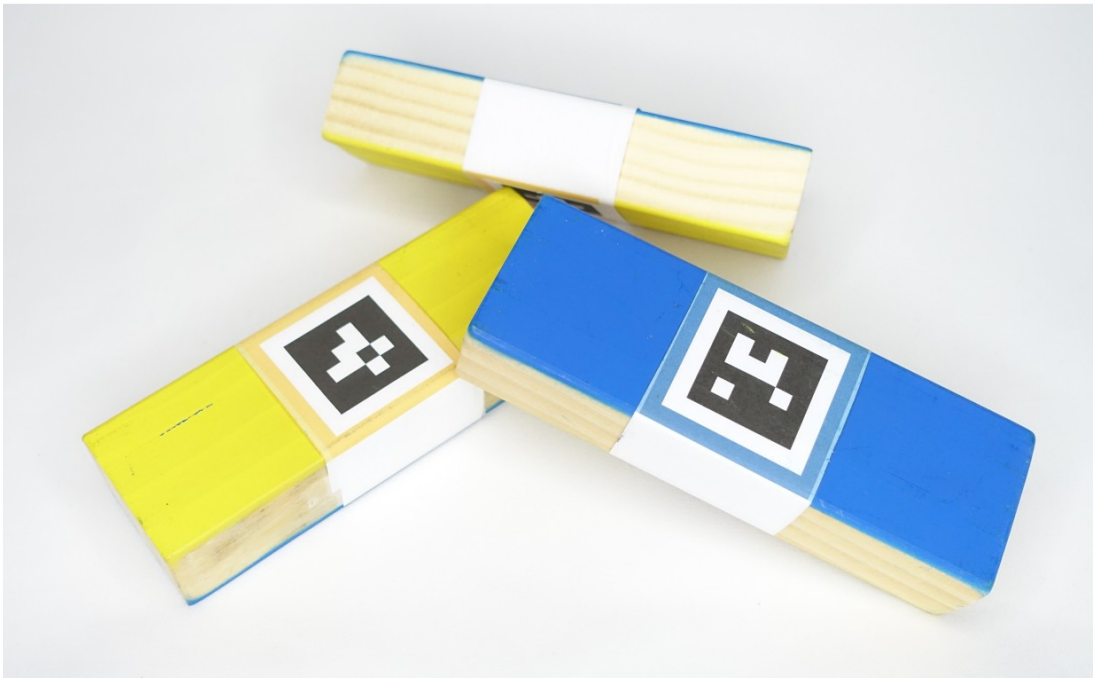


Figure 1.3: Hazelnut crates and game elements.

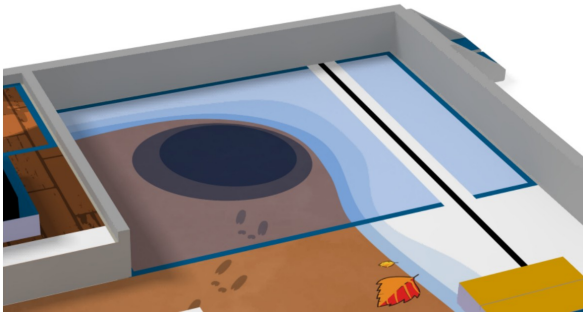


Figure 1.4: *
Starting nest.

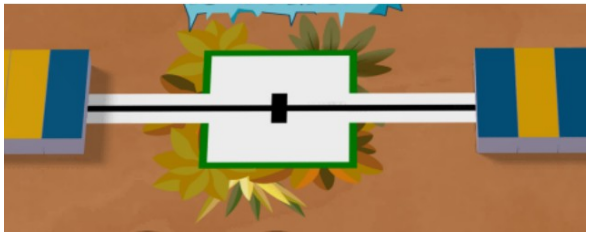


Figure 1.5: *
Pantry.

Figure 1.6: Nest and pantry (granary associated with the game area).

1.3 Main Actions and Scoring System

Action	Points
Hazelnut crate placed in the nest	2 pts / crate
Valid crate placed in a pantry	3 pts / crate
Majority in a pantry	+5 pts
Empty fridge	2 pts
Fridge filled with an empty crate	5 pts
Thermometer cursor position	0 to 10 pts
Robot partially in the nest	5 pts
Robot fully in the nest	+5 pts
Pantry occupied by a PAMI	5 pts
All PAMIs eating at the end	10 pts

Table 1.1: Eurobot 2026 scoring system summary.

1.4 Panorama of Existing Solutions

This section compares international architectures in terms of central computing, sensors, navigation, and communication. The main differences concern middleware choices, sensor types, and path planning methods.

Common architectures fall into two families: centralized solutions, relying on a powerful onboard computer (embedded PC, GPU), and distributed solutions, which delegate certain functions to MCUs. Centralization simplifies software synchronization but increases thermal and energy complexity. Distribution offers better robustness and simpler maintenance, at the cost of more careful communication integration.

Regarding sensors, teams generally combine odometry, LIDAR, and vision. Odometry provides a continuous baseline, LIDAR provides global obstacle perception, and vision enables game element identification. In terms of planning, approaches range from predefined trajectories to online replanning based on A* or its variants.

Software integration choices are equally critical. Some teams favor simple buses (I2C, direct UART) to connect microcontrollers, at the cost of more manual integration into the ROS2 stack. Others rely on ROS2 bridges or distributed systems for a more modular architecture, but requiring more rigorous software configuration.

1.5 Critical Analysis and Positioning

Our solution favors a distributed architecture (RPi5 + STM32 + ESP32), Hybrid A* path planning, and LIDAR/vision perception for an optimal performance/cost trade-off with high modularity.

This strategy isolates real-time control loops on the STM32, delegates planning and strategy to the Raspberry Pi 5, and dedicates the ESP32s to actuators and pneumatics. The choice of Hybrid A* handles kinematic constraints while remaining compatible with

a discretized map. ArUco detection provides punctual pose correction and rapid game element identification.

We chose ArUco detection over simple color detection to improve reliability and robustness in real conditions. Markers offer more stable identification even in the presence of lighting variations or partial occlusions.

For software integration, micro-ROS was chosen over a classic I2C link. This approach enables native integration with ROS2 nodes, clean distributed communication, and better system scalability (nodes can be added without reworking the protocol).

The choice of an SBC + LIDAR architecture is justified by the need for sufficient computing resources for advanced planning and perception. A microcontroller alone (STM32) would be limited for LIDAR processing and dynamic planning. By splitting tasks between the SBC (compute-intensive) and microcontrollers (real-time), we reduce overall computation time and increase robot reactivity.

Our Choice	**Alternative**	**Justification**
ArUco + vision	Color detection only	Robust and reliable identification
micro-ROS	I2C/UART direct	Native ROS2 integration, modularity
SBC + LIDAR	MCU only (STM32)	Advanced planning and global perception
Distributed architecture	Centralized architecture	Isolated real-time, reactivity

Table 1.2: Positioning of our choices against common alternatives.

1.6 Chapter Conclusion

The state of the art confirms the relevance of a modular architecture and an adaptive navigation strategy. The next chapter details the design and the techno-economic study.

2 Design and Techno-Economic Study

2.1 Chapter Introduction

This chapter presents the mechanical, electronic, and software design, as well as the cost analysis.

2.2 Mechanical Design

The mechanical design relies on a three-layer aluminum chassis with 4 mm thickness, providing rigidity and accessibility for module integration. The robot is designed within a footprint of approximately 300 mm × 300 mm × 300 mm. Locomotion uses JSUMO JS7444 wheels and Pololu 4742 DC motors (300 rpm) for the main drive. Secondary actuators use N20 150 rpm motors with integrated encoders, suited for fine movements.

The chassis was selected for its rigidity, lightness, and ease of machining. The power supply is positioned centrally to balance masses and stabilize the center of gravity. Omnidirectional caster wheels (91 Series – Threaded Stud Ball Transfer Units) are added for passive guiding. They support up to 8000 kg, offer low friction (1:5000), and allow speeds up to 2000 mm/s. This configuration improves stability during motion and minimizes energy losses due to friction.

Grasping is handled by a pneumatic solution: each pump feeds four suction cups, and each suction cup has its own solenoid valve. This architecture enables independent control of each grip and rapid adaptation to pick-up/deposit sequences.

Module positioning was designed to maintain a low center of gravity and limit load variations on the drive wheels. Chassis layers are separated to isolate power electronics from perception elements, reducing interference. The layout also facilitates cable routing and quick access for maintenance.

The structure is deliberately modular: each sub-assembly (locomotion, grasping, pneumatics, electronics) can be disassembled independently. This strategy reduces intervention time and simplifies on-site diagnosis. Maintenance was considered by providing direct access to critical elements (battery, regulators, boards, pumps, and solenoid valves), enabling rapid interventions between test runs.

2.2.1 Mechanical Design Methodology

Requirement Analysis

The mechanical design process started with a detailed analysis of the Eurobot rules and the team strategy. The first major decision was to adopt a differential-drive configuration, primarily because the navigation and control stack was already built around differential-drive kinematics. Switching to an omnidirectional platform would have required a new control architecture, longer tuning, and additional validation time.

The propulsion system was based on Pololu 4742 DC motors coupled with JSUMO JS7444 wheels, selected for a balance between size, performance, availability, and cost. The JSUMO wheels provide high grip, which is critical for rapid accelerations. Since accurate odometry is essential in differential drive, the design integrates two spring-loaded encoder modules to maintain continuous ground contact.

Strategic objectives also shaped the mechanical requirements. The robot needed to collect four crates simultaneously rather than sequentially, and to deposit crates in parallel whenever possible. Additionally, the robot had to reorient crates to match the team color pattern, which required a reliable flipping mechanism. Finally, stability constraints imposed a low center of gravity and a wide support base to handle fast accelerations and direction changes.

Concept Development

Several concepts were explored before converging on the final architecture.

Concept A. A forward articulated mechanism collected a full batch and pressed crates against a rotating disc (servo-driven) and a passive support disc for orientation. It was simple and fast, but placed the drive wheels away from the center, causing poor weight distribution and reduced stability.

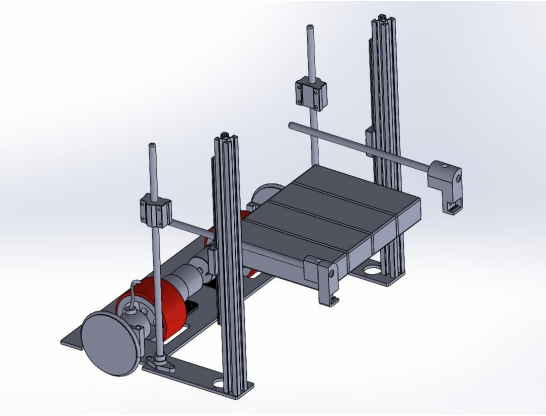


Figure 2.1: Concept A: articulated front collector (initial idea).

Concept B. A two-stage system lifted a batch via a screw-and-nut mechanism, then transferred crates to four independent grippers for rotation and color patterning. While functional, it required many actuators and suffered from high torque demand, making it complex and less reliable.

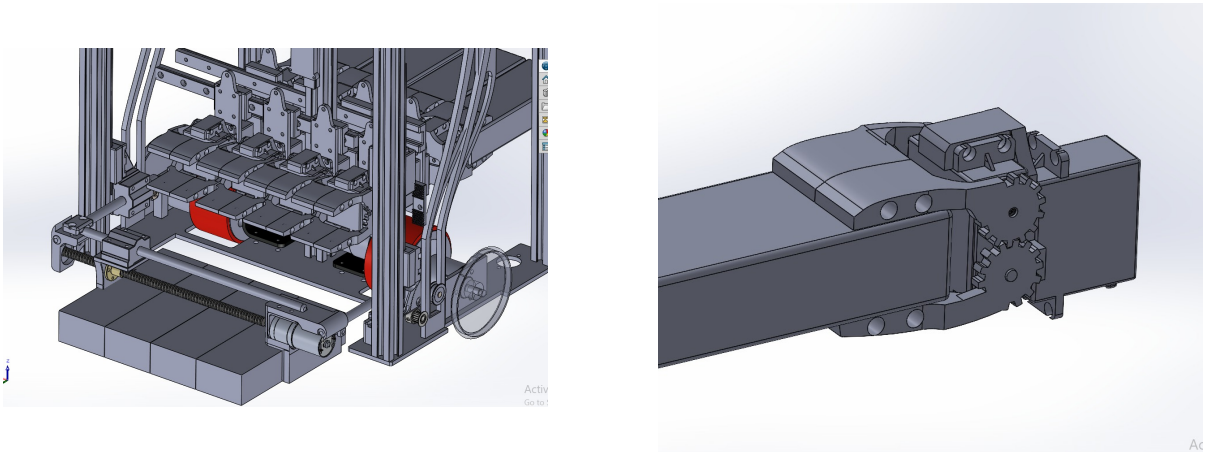


Figure 2.2: Concept B: two-stage lifting and reorientation (prototype views).

Concept C. A gripper on a rotating hinge (about 120°) held crates vertically while two discs oriented each crate. The solution was affordable but mechanically complex, with many moving parts.

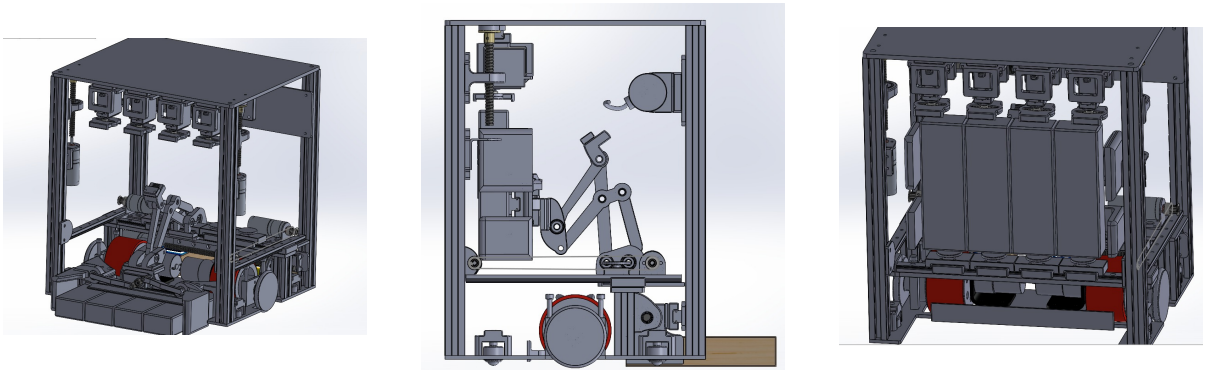
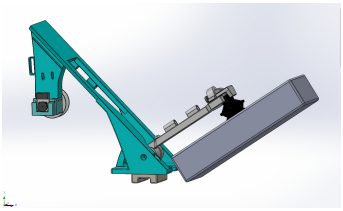


Figure 2.3: Concept C: rotating hinge with disc-based orientation (prototype views).

Concept D. The final concept used a dedicated 3D-printed mechanism per crate with a suction cup. A crate is grabbed at its center, then the arm rotates about 160°, positioning the crate above a 30° incline. Releasing vacuum lets gravity flip the crate into the desired orientation. A centering mechanism ensures proper alignment before suction engagement, and a separation system prevents interference between adjacent crates. This concept reduced mechanical complexity while improving reliability.



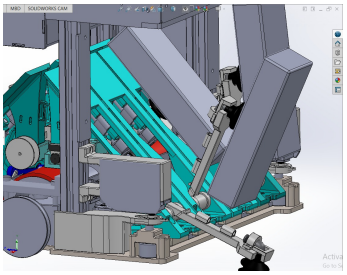


Figure 2.4: Concept D: grasping and pneumatic subsystems (implementation views).

Concept Selection. Concept D provided the best balance between simplicity, manufacturability, reliability, and performance. Its symmetry improved weight distribution, and the reduced number of actuators minimized failure risks during competition.

Design Workflow

Requirement Consolidation. The main requirements were:

- Differential drive with Pololu 4742 motors and JSUMO wheels.
- Two spring-loaded odometry encoders.
- Low center of gravity through strategic component placement.
- Wide chassis geometry for stability.
- Symmetric manipulation modules on both sides of the robot.
- Dedicated electronics compartment with easy maintenance access.

The base uses three stacked 4 mm aluminum plates to reduce the center of gravity. The manipulation system relies on eight independent linear slides (four per side), each driven by an N20 motor and a spring-assisted mechanism. Servo-controlled separation linkages isolate adjacent crates, while centering guides align crates before vacuum engagement.

Electronic Integration Requirements. The mechanical layout includes space for 6 DC-DC converters, one LiPo battery, one power bank, six custom PCBs (2 servo boards, 2 motor boards, 1 pneumatic board, 1 STM32 shield), a Raspberry Pi 5, two vacuum pumps, a USB hub, and the LD19 LIDAR. The compartment supports cable routing, thermal management, and fast access.

CAD Design. The robot was modeled in SolidWorks. The CAD phase covered structural, mechanical, pneumatic, and electronic integration and required about one week of continuous development.

Final Manufacturing. The chassis and aluminum structural elements were CNC machined using detailed 2D drawings. Functional mechanisms were printed using FDM (PLA+) on Bambu Lab X1C and Creality Ender 3 V2 Neo printers. Protective covers and upper panels were laser-cut from MDF sheets.

Assembly. Assembly followed the CAD layout, with attention to cable management, pneumatic routing, component accessibility, and maintenance. This approach reduced assembly time and simplified troubleshooting.

2.2.2 Main Robot Mechanical Design

Chassis Design and Structural Layout

The chassis is a layered low-center-of-gravity structure designed to withstand dynamic loads. The base plate assembly uses three stacked 4 mm machined aluminum plates for rigidity. Four vertical aluminum extrusion profiles connect the base to the upper electronics enclosure, providing modular mounting and torsional stiffness. Upper panels are laser-cut MDF to avoid top-heavy weight.

Heavy components (motors, LiPo battery, aluminum plates) are placed close to the ground plane to keep the center of gravity near the drive axis. Manipulation modules are distributed symmetrically on both sides to avoid weight imbalance during fast turns.

Propulsion and Odometry Integration

The drivetrain is a rear-centered differential drive powered by two Pololu 4742 DC motors. Traction is provided by JSUMO 7444 wheels, whose high-grip rubber minimizes slip during aggressive maneuvers. Odometry is handled by two independent spring-loaded tracking wheels that remain in contact with the ground, absorbing surface irregularities without affecting the main drive wheels.

Crate Manipulation Mechanisms (Mechanism 1)

The primary scoring system consists of four independent pick-and-flip modules. Each module uses a vacuum suction cup fed by two onboard pumps controlled via a pneumatic PCB. The suction cup targets the flat center of the crate, and the bellows geometry tolerates slight misalignments.

The flip sequence uses a passive gravity slide: the arm engages vacuum, rotates the crate about 160° above a 30° guide surface, then releases vacuum so the crate slides and flips into its target orientation. Centering guides (servo-actuated) align crates before pick-up, and separation linkages prevent adjacent crates from interfering during release.

Electronics Enclosure and Maintenance Integration

The upper enclosure houses the computing and power distribution systems in a top-loading layout. A hinged cover provides full access for rapid maintenance. Ventilation cutouts and an integrated fan dissipate heat from converters and drivers. Internal routing channels separate power and data cables to reduce EMI. Boards (Raspberry Pi 5, STM32 shield, motor and pneumatic PCBs) are spaced to prevent thermal pooling and simplify service operations.

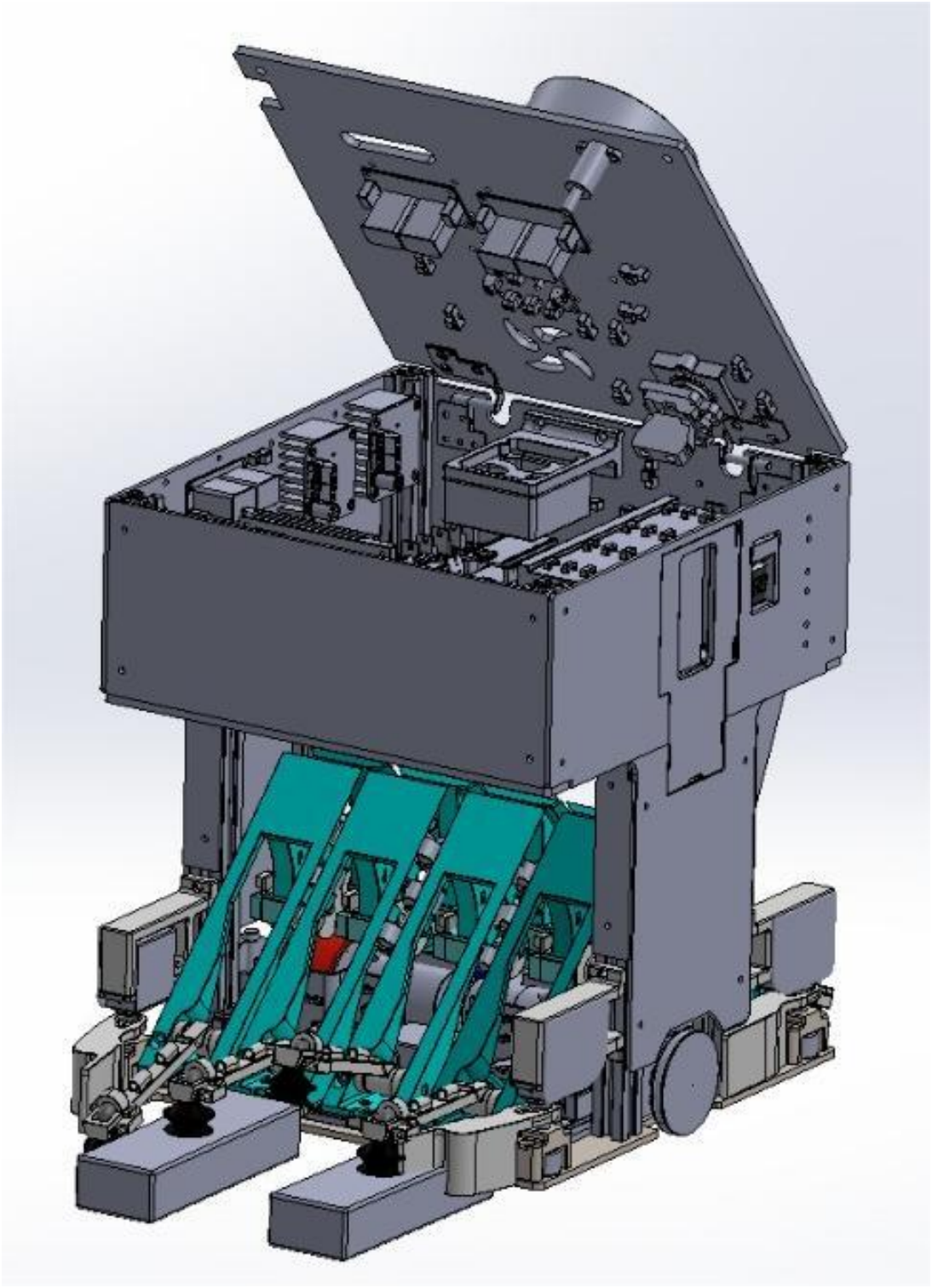


Figure 2.5: Isometric CAD view of the robot.

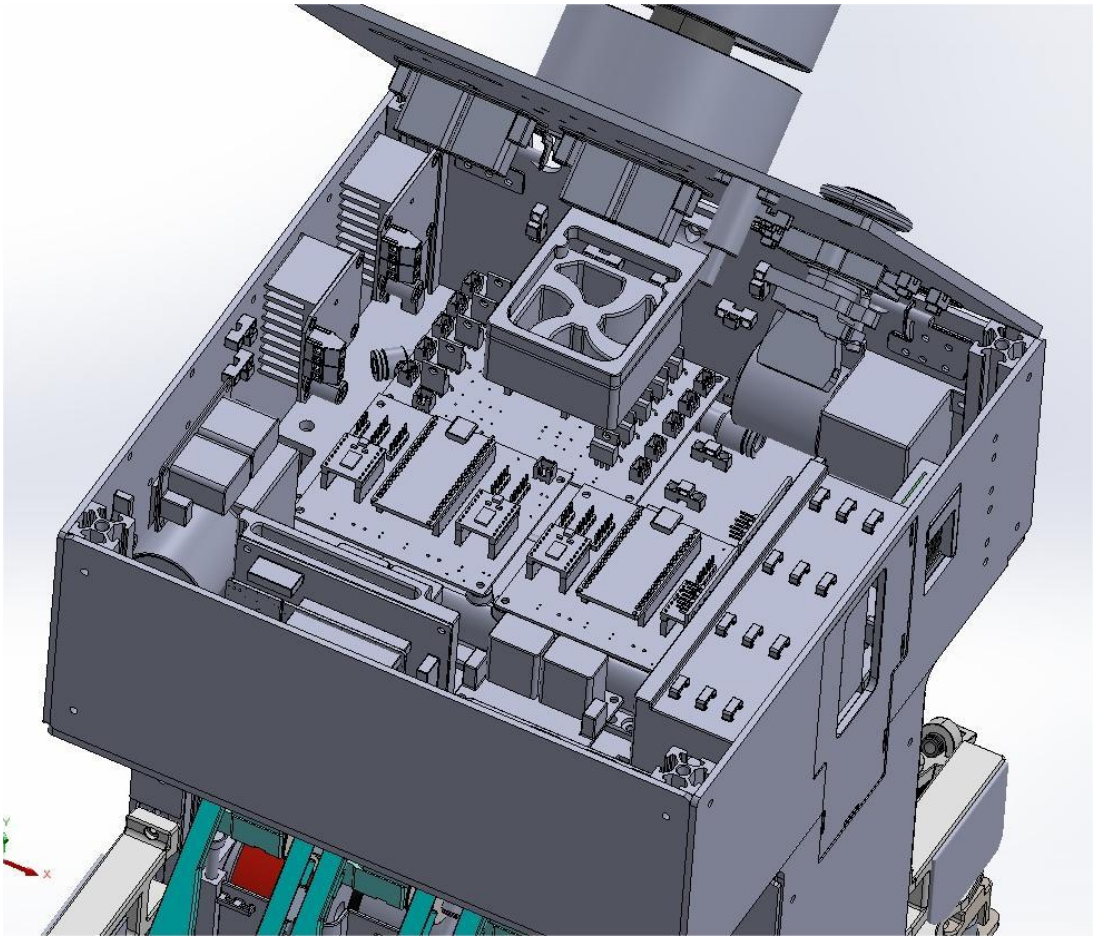


Figure 2.6: Top CAD view of the robot.

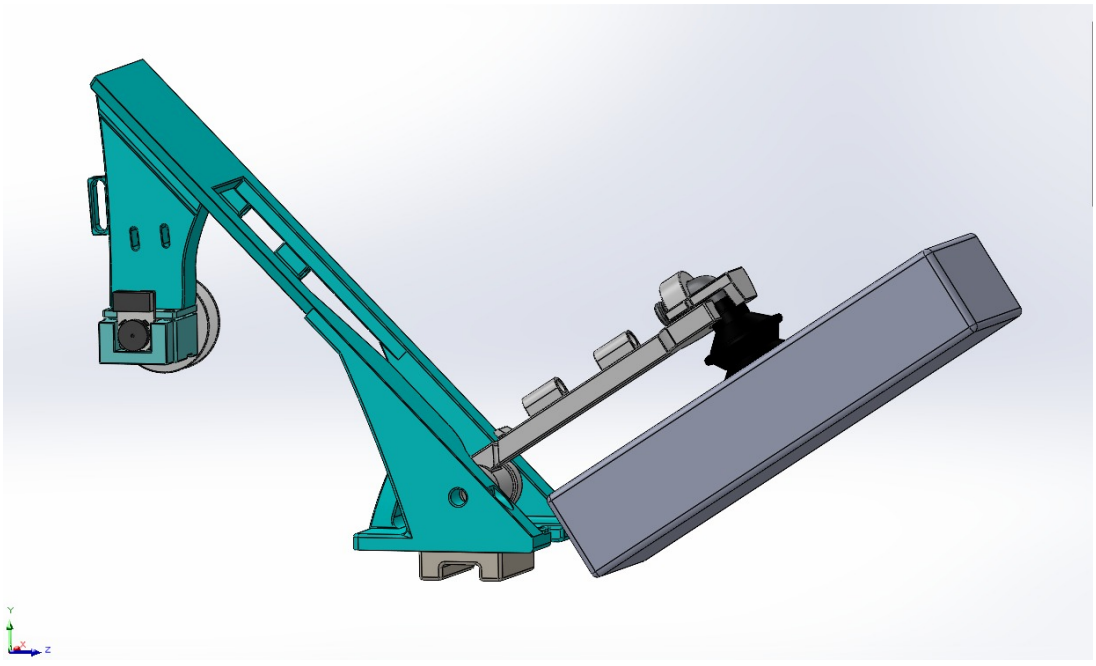


Figure 2.7: Grasping system (gripper/suction cup).

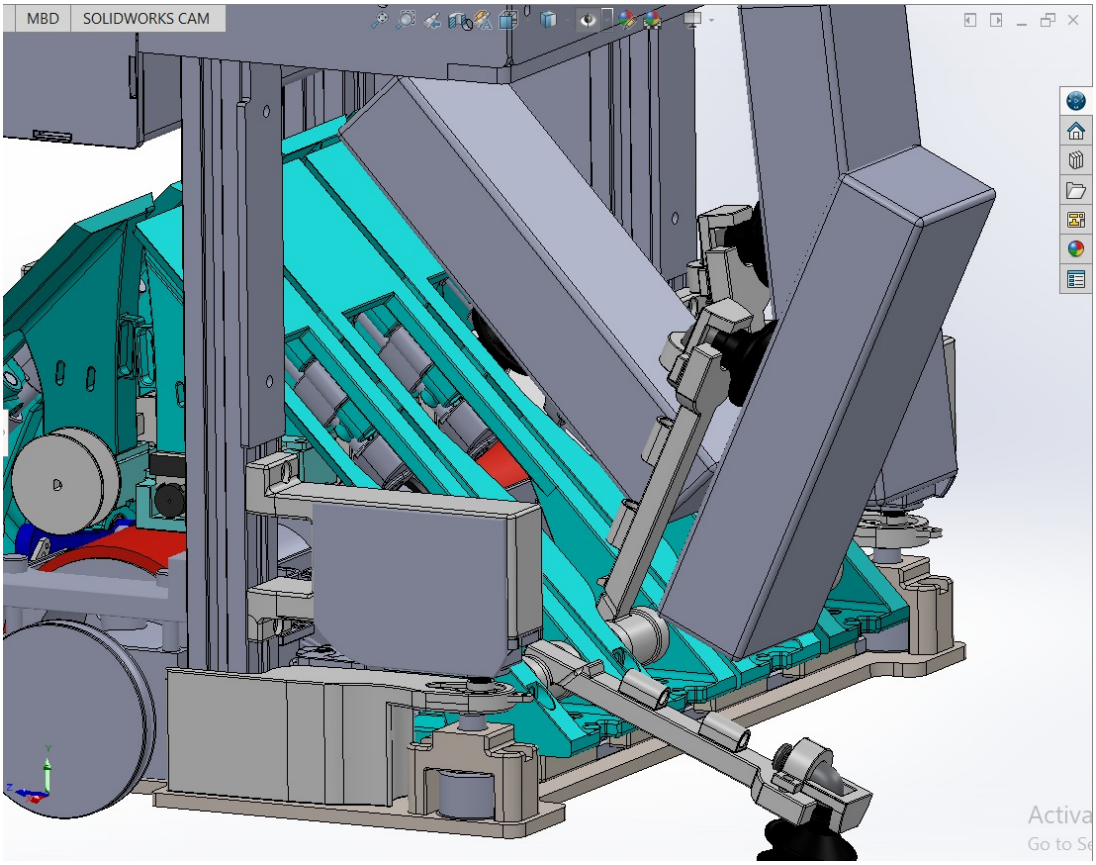


Figure 2.8: Mounted pneumatic system.

2.3 Electronic Design

- Actuator drivers: TB6612FNG.
- Main motor drivers: IBT-4.
- Optical encoders: 3000 ticks/rev.
- Power supply: LiPo 5000 mAh with five voltage regulators to isolate current rails.

The TB6612FNG was chosen for its compact footprint and lower losses compared to classic H-bridge circuits, while the IBT-4 is reserved for higher-current loads. PCB1/PCB2 are ESP32 + TB6612 shields for actuators (4 N20 motors with integrated encoders + 3 servos per board). PCB3 drives the pneumatic system via MOSFETs (pumps and solenoid valves). Two separate servo shields protect the microcontrollers and isolate servo commands.

The power supply is based on a LiPo 5000 mAh battery, distributed through five regulators to separate sensitive rails (logic/vision) from impulsive loads (motors/pumps). This separation limits voltage drops and EMI disturbances that could cause MCU resets. Cable sizing and filtering choices were prioritized to guarantee stability during a match.

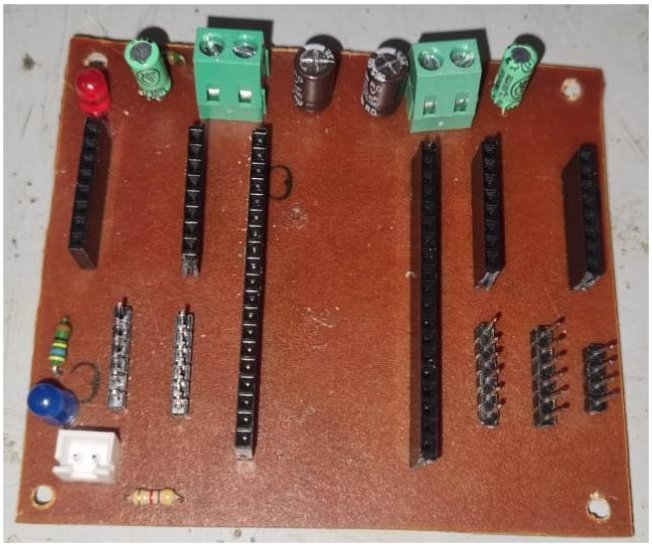


Figure 2.9: PCB1/2 schematic (ESP32 + TB6612FNG drivers).

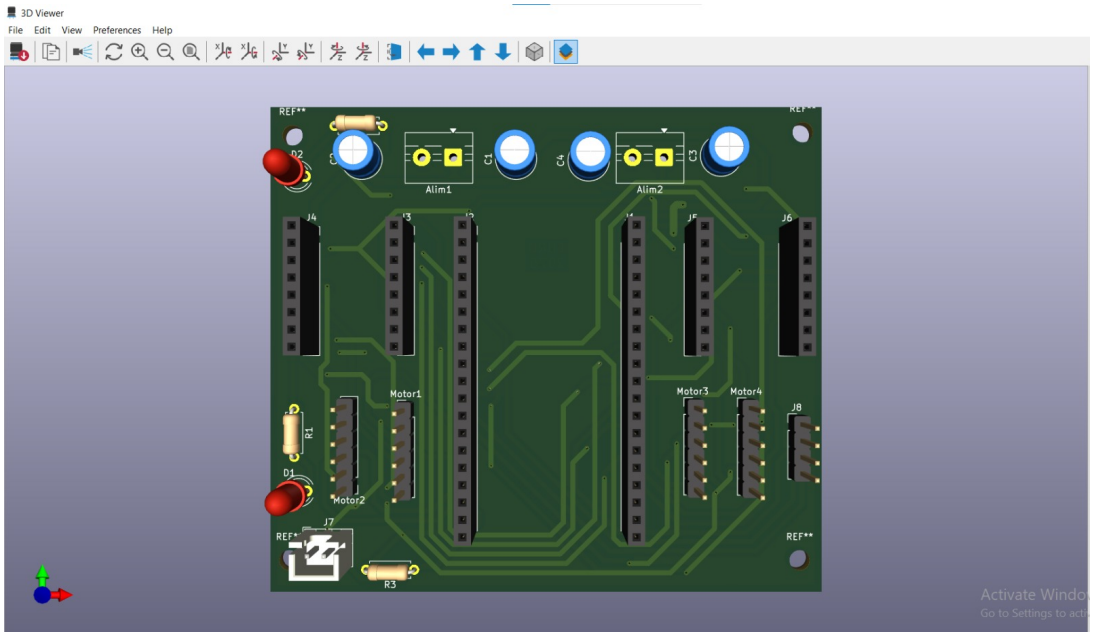


Figure 2.10: 3D view of PCB1 and PCB2.



Figure 2.11: PCB3 schematic (MOSFETs for pumps/solenoid valves).

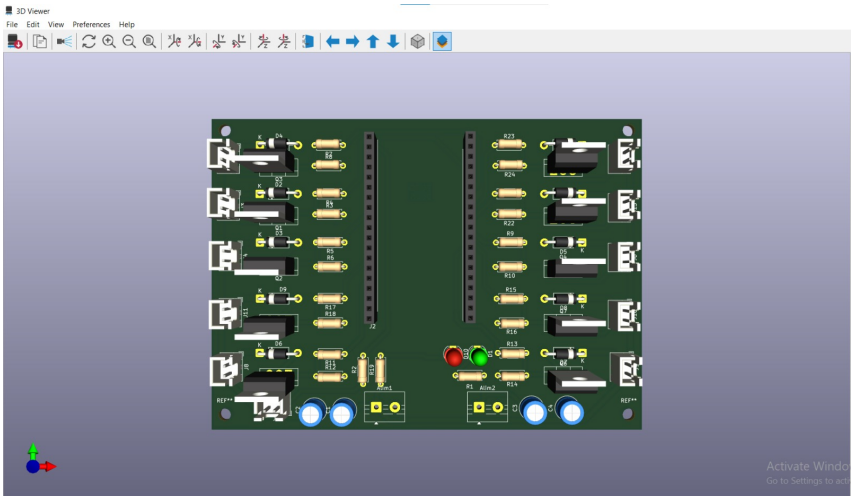


Figure 2.12: 3D view of PCB3.

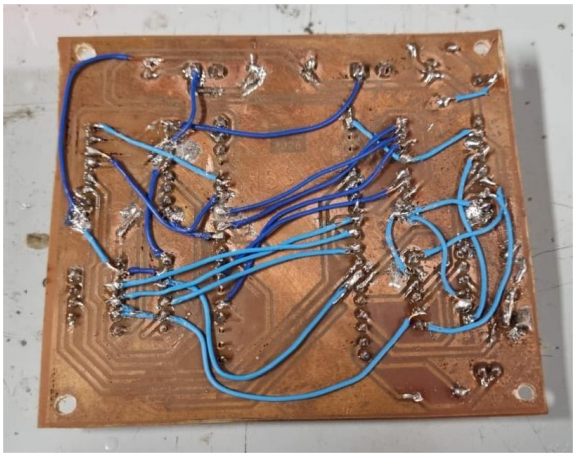


Figure 2.13: Photo of soldered PCBs (front side).

2.4 Software Design

The global architecture is integrated into the software stack. A Raspberry Pi 5 (ROS2 Jazzy) handles strategy and path planning, a STM32F446RE manages real-time navigation, and three ESP32 boards control actuators and the pneumatic system. Communication is handled through micro-ROS.

Information flows follow a clear chain: perception (LIDAR, vision) leads to map construction, objective planning, trajectory generation, and finally low-level commands. The separation of responsibilities facilitates debugging and maintenance. The ESP32s provide fast actuator cycles, while the STM32 maintains navigation stability.

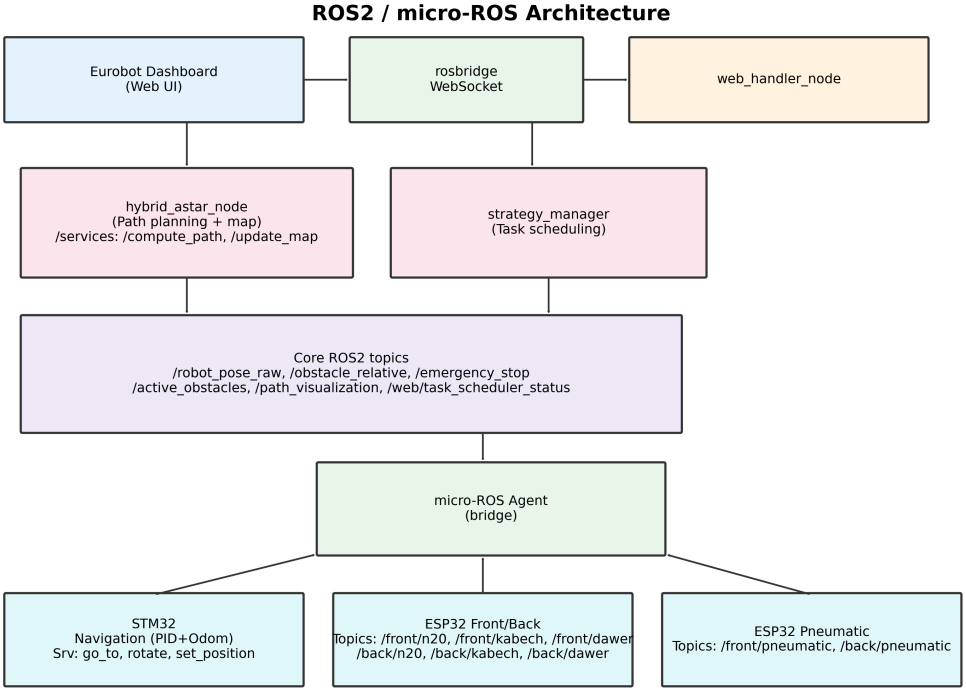


Figure 2.14: Block diagram of the ROS2 / micro-ROS architecture.

2.4.1 ROS2 Architecture

The software architecture is built around ROS2 and micro-ROS. The Raspberry Pi 5 hosts planning services, match strategy, and the web bridge. The MCUs handle real-time control loops.

Node	Role and main interfaces
hybrid_astar_node	Services: /compute_path, /update_map, /reset_map, /clear_map, /start_game, /put_down, /get_batch_color_pattern, /delete_batch. Published: /active_obstacles.
web_handler_node	Rosbridge gateway. Sub: /web/start_pose, /web/goal_pose, /web/enemy_pose, /web/compute_path, /web/get_batch_color. Pub: /web/path_result, /web/task_scheduler_status.
map_manager	Sub: /obstacles/clear_all, /obstacle_relative, /aruco_mapper/all_batches. Pub: /obstacles/current, /path_visualization.
strategy_manager	Sub: /robot_pose_raw, /aruco_mapper/state_robots. Pub: /web/task_scheduler_status.
pathfinder_client	Client: /compute_path, /update_map, /delete_batch. Sub: /robot_pose_raw, /obstacle_relative.
front_systems _controller	Pub: /front/n20, /front/kabech, /front/dawer. Client: /front/isIDLE.
back_systems _controller	Pub: /back/n20, /back/kabech, /back/dawer. Client: /back/isIDLE.
pneumatic_controller	Pub: /front/pneumatic, /back/pneumatic.
lidar	Sub: /emergency_stop, /robot_pose_raw, /obstacle_relative.
simple_map_node	Sub: /robot_pose_raw, /obstacle_relative, /obstacle_relative_speed, /active_obstacles.
pami_controller	Pub: /pami{n}/strategy, /pami{n}/start.

Table 2.1: ROS2 nodes and their main interfaces.

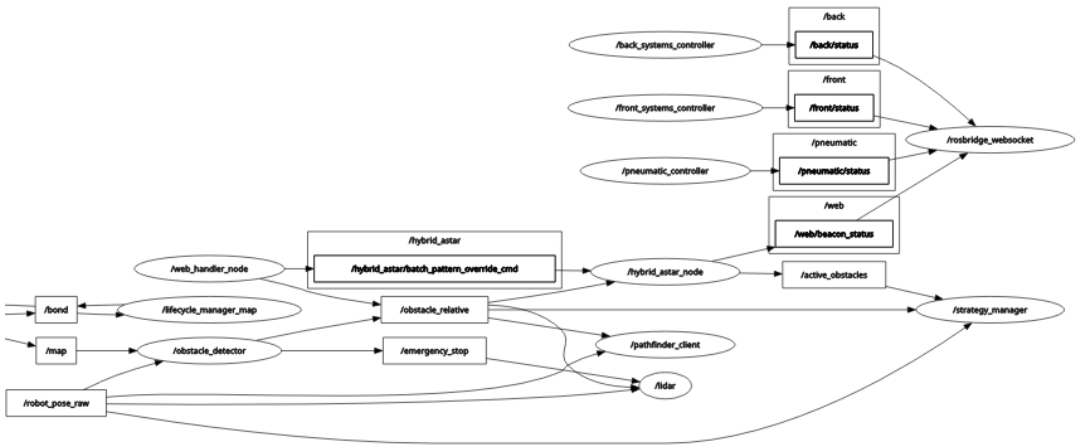


Figure 2.15: ROS2 node graph (rqt_graph).

The software layer includes a state-machine-based match strategy (preparation, collection, deposit, return). Each state triggers service calls and trajectory goals. Actuator feedback is used to confirm action completion before transitioning to the next step.

The distributed architecture isolates time-critical processing (control loops, actuators) from computationally heavy tasks (planning, perception). This separation prevents software blocking and guarantees deterministic responses during grasping and deposit phases.

2.4.2 Match Strategy and Task Scheduling

The strategy is built as a task scheduler that is updated according to the game state and remaining time. Priority objectives are dynamically re-evaluated to avoid conflicts with the opponent and maximize score. This logic is connected to the planner through service calls and state feedback, enabling robust action chaining.

The state table relies on conditional transitions: an action is considered complete only when the actuator confirms execution and the robot pose is validated. This method limits cumulative errors and ensures reliable execution even in the presence of disturbances.

Target selection is performed by a Task Scheduler based on a weighted global score. For each task (pick-up/deposit), a score is computed from several criteria: distance to the target, estimated enemy objective, number of obstacles between the robot and the target, proximity of the enemy, distance to the nest, crate color (opposition), and available/loaded robot face. Remaining time influences certain terms (nest return), and priority weights favor strategic actions. Tasks are then sorted by decreasing score, and the best one is selected as the current objective.

2.4.3 Navigation and Localization

The STM32 runs PID loops and computes odometry. Localization is assisted by vision (ArUco markers) to estimate the position of game elements and perform XY correction. Color detection helps discriminate objects during pick-up and deposit phases.

Odometry relies on encoders and a differential kinematics model. PID loops are separated into position and velocity to achieve a fast response without excessive oscillations. Visual corrections are applied punctually to limit accumulated drift while avoiding trajectory destabilization.

2.4.4 PID Motor Control (STM32)

Motor control is structured around two distinct PID loops: velocity PID and position PID. The derivative term is disabled on the velocity loop to avoid high-frequency noise. The main gains are:

Loop	**Gains**
Velocity PID	$k_{pS}^{R/L} = 15$, $k_{iS}^{R/L} = 200$, $k_{dS}^{R/L} = 0$
Position PID	$k_{pP}^{R/L} = 15$, $k_{iP}^{R/L} = 75$, $k_{dP}^{R/L} = 1.4$

Table 2.2: PID gains used for navigation.

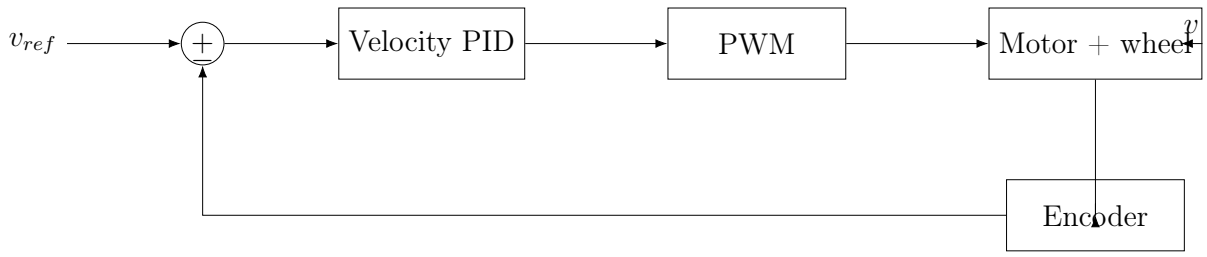


Figure 2.16: Velocity PID control loop (inner loop).

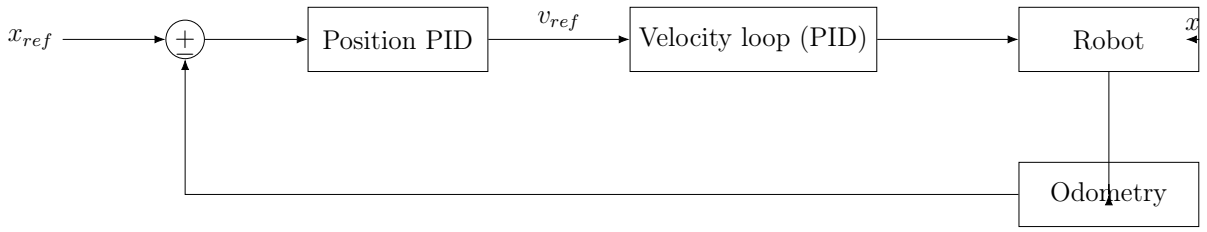


Figure 2.17: Position PID control loop (outer loop).

Velocities are estimated from encoders using an exponential filter (e.g., 0.7/0.3) to reduce jitter. Accelerations are also filtered with coefficients close to 0.998. The controller integrates minimum PWM thresholds, offsets, stall protection, and operating states (acceleration, cruise, controlled braking, emergency braking). The system operates in *SPEED* or *POSITION* mode depending on the movement phase.

2.4.5 Odometry and Pose Correction

Odometry is computed from right/left encoder readings. At each update, the average distance d_C is estimated and the pose is integrated:

$$x_{t+1} = x_t + d_C \cdot \cos(\theta) \tag{2.1}$$

$$y_{t+1} = y_t + d_C \cdot \sin(\theta) \tag{2.2}$$

The orientation θ is obtained from the wheel distance difference and normalized to $[-\pi, \pi]$. The code supports ArUco correction via the *set_position* service: when a tag is detected at the front or rear, the robot's position is recalculated from the tag position and mounting offsets. This correction reduces odometric drift over long sequences.

2.4.6 Navigation and Trajectory Generation

Navigation is based on a state machine (IDLE, GOTO, ROTATE, CREEP, BEZIER) and a velocity ramp profile (acceleration, cruise, braking). For each movement, the *ramping_acceleration* and *breaking_acceleration* parameters define the velocity ramp-up and ramp-down.

Main functions include *go_to()* and *move_forward_full()* for direct trajectories, *rotate() / orientate()* for angular alignment, *bezier_curve()* for smooth trajectories, and *creep_to()* for a safe slow approach. A trajectory correction term (angle and lateral drift correction) is applied continuously to stay aligned towards the target. An emergency stop is triggered if velocity remains zero despite high PWM (collision or stall). The *hybrid_arrival_time()* function combines a theoretical profile with a real-velocity estimate to improve stopping accuracy.

2.4.7 LIDAR Perception and Obstacle Management

The LIDAR LD19 provides 360° detection at 10 Hz. Points are transformed into the robot frame and integrated into the local map. A safety zone is maintained around obstacles (approximately 30 cm in simulation) to prevent collisions during replanning. Detection of moving obstacles is taken into account by the strategy to adapt objectives and limit the risk of deadlock.

In LIDAR_EUROBOT, filtering uses a pipeline of clustering + map validation + exponential smoothing. Scans are converted to points, then grouped into clusters based on a distance threshold, a minimum size, and a maximum diameter. Each cluster is then validated against the occupancy map (only points in free space are kept), and clusters that are too large or too small are rejected. Once tracked, an obstacle's position is smoothed using an Exponential Moving Average (EMA) in the map frame, with stronger filtering during rotations to avoid false displacements due to pose errors. A short history is maintained to estimate speed and trajectory and to activate safety measures when necessary.

2.4.8 Path Planning and Occupancy Grid

Path planning uses Hybrid A*. The occupancy grid is divided into cells of fixed resolution. Each cell is either free or occupied, taking into account the robot's footprint with a safety margin.

Hybrid A* combines a discrete graph with continuous orientation expansion, enabling the generation of feasible trajectories for a non-holonomic robot. The trajectory cost integrates path length, direction changes, and proximity to obstacles. Replanning is triggered when a moving obstacle is detected or when an objective becomes unreachable.

Planning occurs in two steps: a grid search to obtain a candidate path, then validation against kinematic constraints (curvature radius, final orientations). This approach eliminates infeasible trajectories and guarantees smoother execution at the low-level control layer.

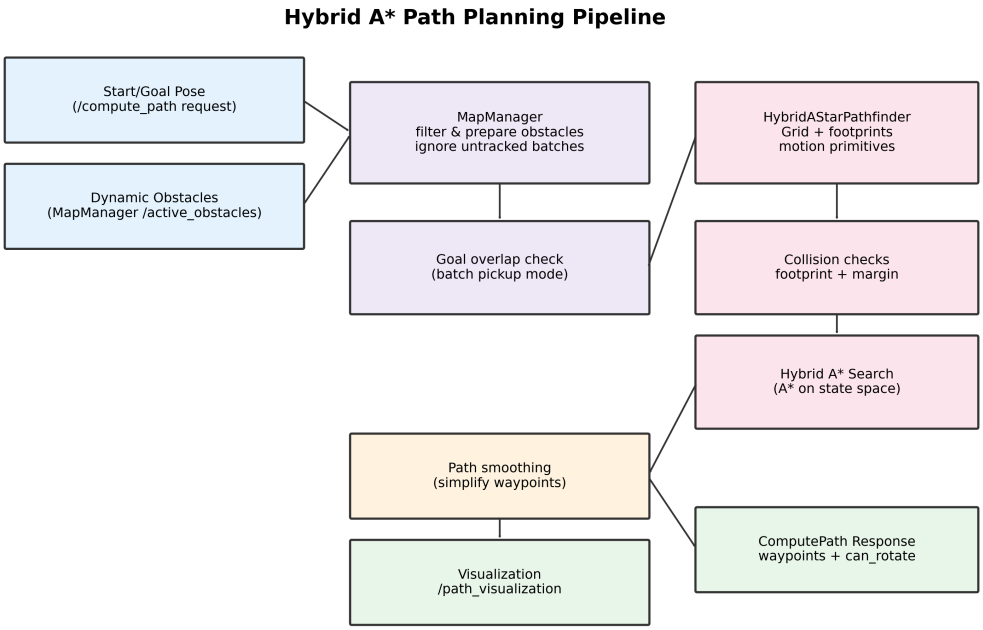


Figure 2.18: Hybrid A* path planning block diagram.



Figure 2.19: Occupancy Grid (RViz).

2.4.9 Vision and Detection

The Raspberry Pi camera detects colors and ArUco markers to identify game elements and improve grasp reliability.

The vision pipeline follows three main steps: calibration, marker detection, then relative position estimation. The information is fused with odometry to stabilize localization and adapt the strategy. Color detection is used to classify game elements and confirm grasping actions.

2.4.10 Embedded AI for Pickability Prediction

The `aruco_mapper_ros` node integrates a pickability prediction model based on OpenVINO. Model artifacts are stored in `model_artifacts/openvino_model/ir/` (`batch_corrector.xml` and `batch_corrector.bin`), and input normalization uses `scaler.pkl` loaded with `joblib`. When `pickability_enable=true`, the node loads the OpenVINO model on the CPU, applies the scaler, and runs inference only for FX batches with exactly four members.

The model input is a 33-dimensional feature vector composed of:

- Batch center (x, y).
- Orientation encoded as $\sin(\phi)$ and $\cos(\phi)$.
- Average adjacent distance between members.
- Four members $\times$ (x, y, z, yaw).
- One-hot of the known batch ID (10 classes: f1...f10).
- One-hot of the team area (blue/yellow).

After inference, the model outputs a pickability probability. If the value exceeds the configured threshold (default 0.72), the batch state is tagged with suffix `_P`; otherwise `_NP`. This allows the mapping pipeline to classify batches as pickable versus non-pickable while keeping the geometric tracking unchanged. If the model or scaler fails to load, pickability is automatically disabled and the system falls back to the classical (non-AI) mode.

The inference pipeline is intentionally conservative. It only runs when a batch is fully observed (exactly four members) and keeps the geometric estimation unchanged. The AI output is therefore used as an additional semantic label rather than a substitute for localization. This separation preserves deterministic behavior while still benefiting from learned classification.

From an operational standpoint, the threshold provides a tunable trade-off between aggressive picking and safe execution. A higher threshold reduces the risk of failed grasps at the cost of fewer candidate batches, while a lower threshold favors coverage but may increase failed attempts. The threshold is kept configurable to adapt to lighting and surface conditions without changing the model itself.

Finally, the OpenVINO integration on CPU ensures predictable deployment on the Raspberry Pi 5, with a lightweight runtime and a clear fallback path. This design keeps the system robust: when AI is not available, the robot continues to operate using classical geometric tracking and rule-based decisions.

The pickability classifier operates as a deterministic pipeline in which a fixed-length feature vector is extracted from each FX batch (geometric center, orientation encoded as $\sin\phi/\cos\phi$, adjacency distance, per-member pose features, and categorical one-hot encodings). This raw feature vector is first normalized using the stored scaler to match the training distribution. The normalized vector is then passed to the OpenVINO inference engine, which outputs a pickability probability. Finally, the probability is thresholded to produce a discrete label appended to the batch state (`_P` for pickable, `_NP` for non-pickable).

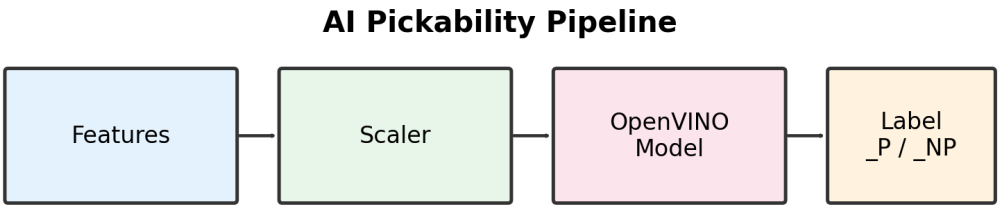


Figure 2.20: AI pickability pipeline (feature extraction, normalization, inference, labeling).

2.5 Techno-Economic Study

The following Bill of Materials (BOM) lists all major components used in the robot, categorized by subsystem.

Component	Qty	Role
Computing & Communication		
Raspberry Pi 5 (8GB)	1	Central computing, ROS2, strategy
STM32F446RE (Nucleo)	1	Real-time navigation PID
ESP32 DevKit	3	Actuator control + pneumatics
Perception		
LIDAR LD19	1	360° obstacle detection
Raspberry Pi Camera v3	2	Vision, ArUco localization
Optical encoders 3000 ticks/rev	2	Odometry (main motors)
Locomotion		
Pololu motors (navigation)	2	Main drive motors
JSUMO JS7444 wheels	2	Drive wheels
IBT-4 motor driver	1	Main motor driving
Actuators		
Pololu N20 motors (actuators)	8	System actuators (arm, conveyor)
Servo motors	6	Gripper and positioning actuators
TB6612FNG motor driver	2	N20 actuator driving (PCB1/2)
Pneumatic System		
Vacuum pump	2	Suction cup air supply
Solenoid valve (electrovane)	8	Independent suction cup control
N-channel MOSFET	10	Pump and valve switching (PCB3)
Power Supply		
LiPo 5000 mAh battery	1	Main robot power supply
Powerbank 20000 mAh	1	Raspberry Pi 5 dedicated supply
Voltage regulators (×5)	1 set	Rail isolation (logic/power)
Electronics		
Custom PCB (ESP32 shield)	2	PCB1/2 – actuator control
Custom PCB (pneumatic)	1	PCB3 – MOSFET switching
Servo shield PCB	2	MCU protection + servo command

Table 2.3: Bill of Materials (BOM) – complete component list.

The techno-economic analysis highlights a favorable performance-to-cost trade-off. Components were selected for their availability, reliability, and compatibility with modular integration. The separation of power rails between logic (Powerbank for RPi5) and actuation (LiPo for motors and pumps) prevents EMI-induced resets and ensures system stability throughout the match.

2.6 Chapter Conclusion

The chosen architecture guarantees modularity and reactivity. The next chapter validates these choices through simulation before physical implementation.

3 Simulation Before Implementation

3.1 Chapter Introduction

Simulations allow us to verify feasibility, tune parameters, and reduce risks before physical integration.

3.2 Simulation Environment

The simulation is carried out under ROS2 Jazzy with RViz2 (without Gazebo). Trajectories, maps, obstacles, and robot state are visualized in real time. Interaction is also provided via the Eurobot Dashboard website through rosbridge, enabling monitoring of robot state and planning.

Test maps are generated from an occupancy grid and game zones. Dynamic obstacles are injected to simulate a moving opponent. This method provides fast algorithm validation without the overhead of a full physics simulation.

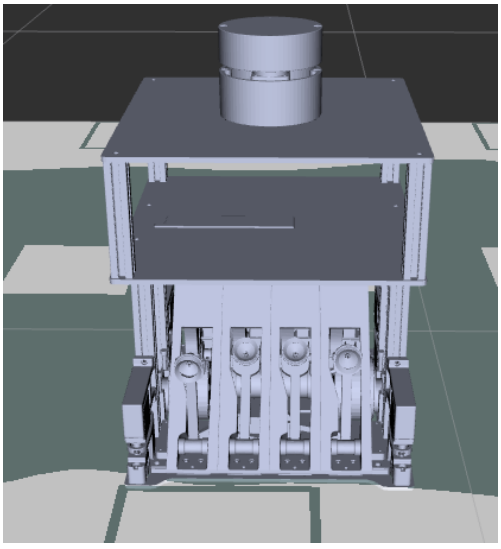


Figure 3.1: RViz2: robot on the Eurobot 2026 map.

3.3 Navigation and Path Planning Simulation

The Hybrid A* path planner is tested on several scenarios with fixed and mobile opponents. The robot maintains a safety distance of approximately 30 cm around obstacles. The average computation time for a 1.5 m trajectory with obstacles is approximately 80 ms. Dynamic strategy target selection takes approximately 1 ms, enabling efficient in-match replanning.

Tests show a stable trajectory with the safety distance respected around forbidden zones. Replanning occurs without visible trajectory discontinuity, limiting oscillations and time losses. Validation in RViz2 also allows verifying topic consistency and synchronization between sensors and planner.

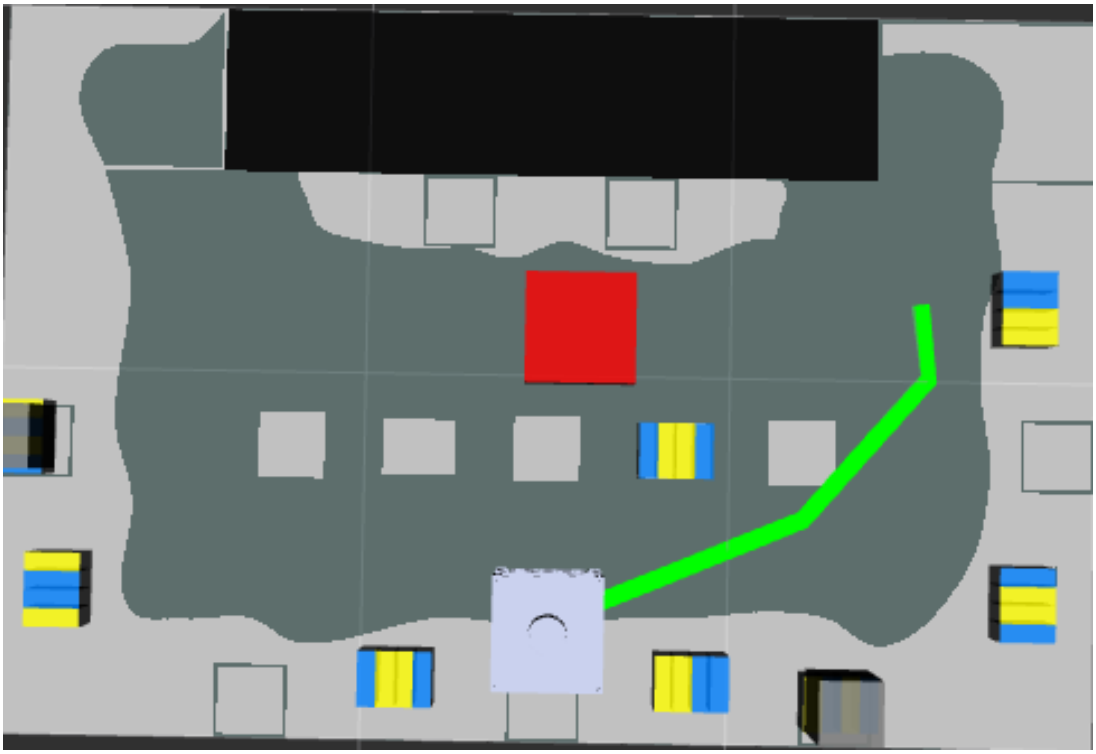


Figure 3.2: Planned trajectory (Hybrid A* on map).

3.4 Control Loop Simulation

Step responses of the navigation and actuator PID loops are analyzed to achieve a good balance between stability and responsiveness.

PID parameters were tuned to minimize response time while avoiding oscillations. Tests with composite trajectories (successive segments and rotations) were used to evaluate the stability of the navigation-actuator system as a whole.

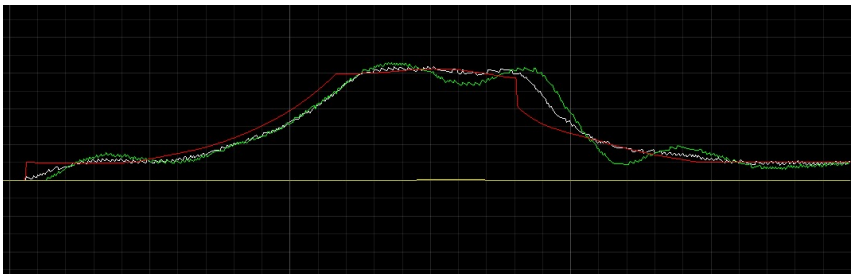


Figure 3.3: Navigation PID curves (step response).

3.5 Chapter Conclusion

Simulations validate the control and planning choices. The next chapter details the physical implementation and testing on the robot.

4 Implementation, Testing and Validation

4.1 Chapter Introduction

This section presents the mechanical and electronic integration, unit tests, and competition evaluation.

4.2 Physical Implementation

Several prototypes (MDF and 3D printing) were used to validate mechanical solutions before final assembly. The final integration ensures good accessibility to wiring and electronic modules.

The assembly phase incorporated progressive verification: chassis assembly, motor integration, power wiring, then sensor and control board integration. Each step was accompanied by functional tests to limit coupling errors.

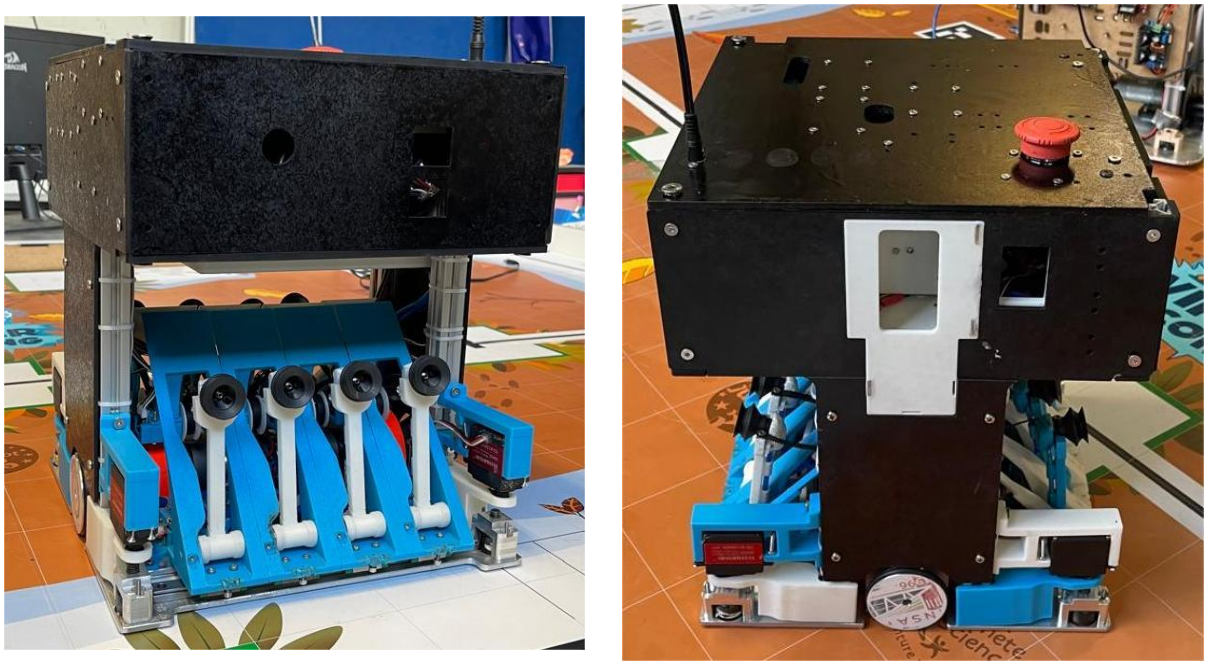


Figure 4.1: Assembled robot (front and side views).

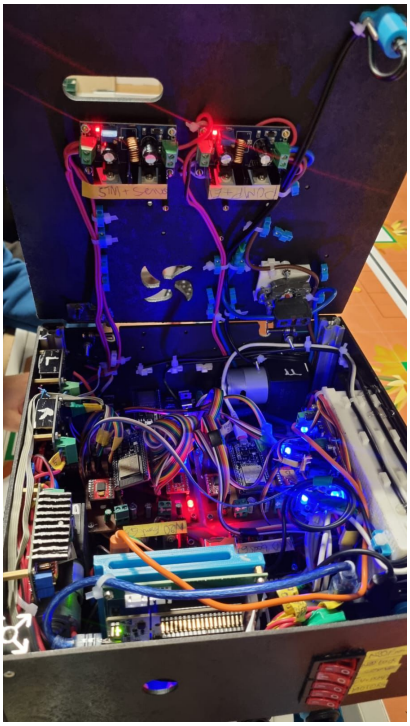


Figure 4.2: Internal wiring.

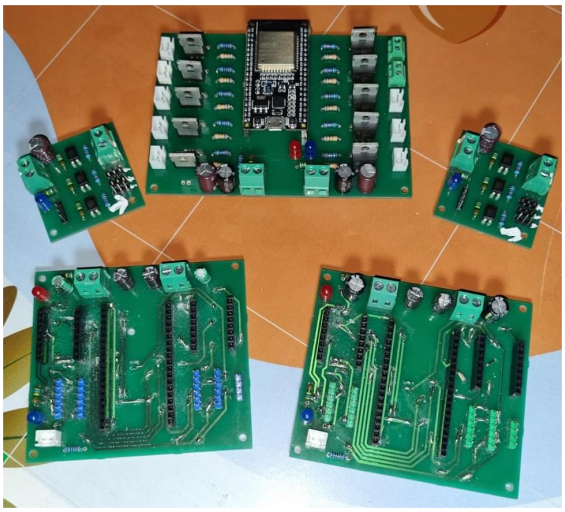


Figure 4.3: PCBs photo.

4.3 Software Deployment

The ROS2 system is deployed on the Raspberry Pi 5. The MCUs are flashed with micro-ROS to ensure real-time communication.

The software configuration includes launch scripts to initialize essential nodes and verify connectivity. Supervision via rqt and the web dashboard provides a clear visualization of states and errors.

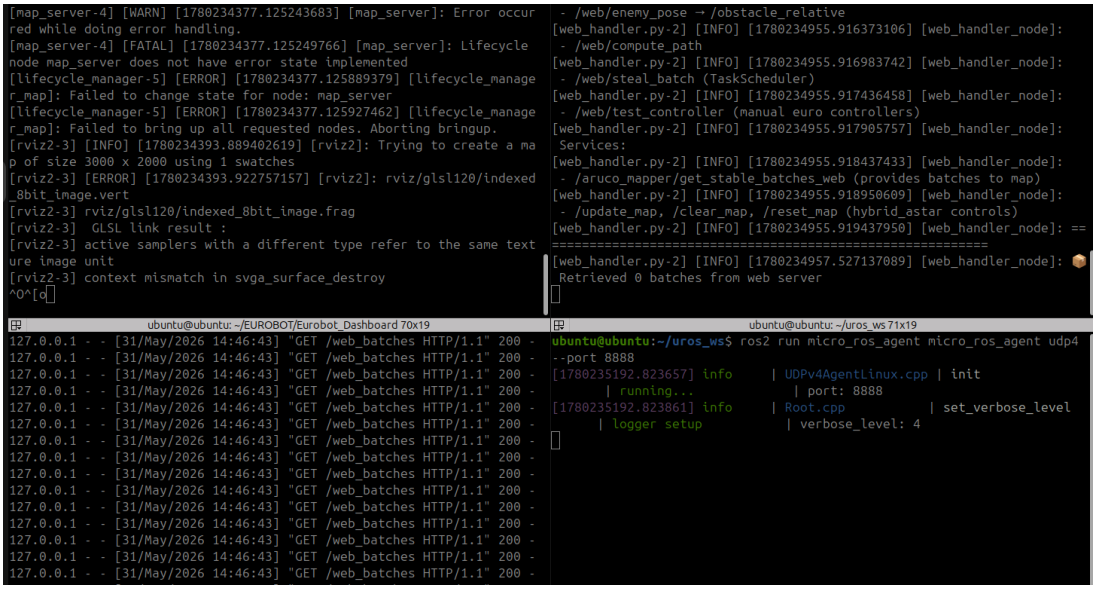


Figure 4.4: ROS2 terminal screenshot (active topics, latency).

4.4 Unit Tests

Tests show a navigation precision of 3 mm after 100 seconds of navigation, validating the quality of the control system.

Test	**Measurement**	**Result**
Linear navigation 100 s	Final error	3 mm
90° rotation	Angular error	0.02°
LIDAR LD19	Acquisition frequency	10 Hz

Table 4.1: Main unit test results.

Additional tests focused on actuator reactivity, servo stability, and consistency of micro-ROS state feedback. These tests validated the synchronization between commands and execution.

4.5 Integration Tests and Competition

A complete match scenario shows that the robot completes its actions in approximately 60 seconds, i.e., 40 seconds before the 100-second limit. The team achieved 1st place in the national qualifications and represented Tunisia in France, reaching 2nd place worldwide across the five series of the international final.

Full integration confirmed the coherence between strategy, planning, and actuators. Transitions between states (collection, deposit, return) occur without deadlock, which explains the time margin observed during matches.

4.6 Observed Limitations

The LIDAR frequency (10 Hz) remains a limiting factor at a speed of 1 m/s. At this speed, a moving opponent moves significantly between two measurements, making position estimation less stable and fine avoidance more difficult.

Potential improvements include increasing perception frequency or sensor fusion (vision + LIDAR) to more accurately estimate opponent velocity. A local slow-down strategy when the opponent's position is uncertain can also enhance safety.



Figure 4.5: Competition photos – Tunisia (national qualifications).



Figure 4.6: Competition photos – France (international final).

Classement final à l'issue de la 6ème série:

Rang	Équipe	Cumul des points
1er	DIT Robotics	637
2nd	AEROBOTIX INSAT	615
3ème	Hyperion Robotics	558
4ème	TURAG	538
5ème	TeamAuto	523
6ème	NTHU DIT	514
7ème	Memristor	514
8ème	µ	489
9ème	Bobst	391
10ème	UCLouvain - Quinte Nut Royale	383
11ème	MONS SNOW	373
12ème	µ Mini	367
13ème	Libres	348
14ème	RTFM	273
15ème	MAI	266
16ème	Universitat Politècnica De València	259
17ème	HMU Robotics Club	242
18ème	Bodensee RobotiX	238
19ème	Universitat De Vic	183
20ème	IEEE RAS INSAT	134
21ème	NTU Maker	131
22ème	TUCaN Robotics	101
23ème		96
24ème	TALEB-BOT	82
25ème	HYPERSONIC	74
26ème	UNICT-TEAM	74
27ème		71

4.7 Chapter Conclusion

Tests confirm the robot's reliability and the validity of the technical choices. The results show that the distributed architecture achieves a stable real-time behavior while preserving high-level adaptability. The integration between perception, planning, and actuation is coherent, and the system maintains its performance under match constraints (time pressure and opponent interactions).

Beyond raw scores, the chapter demonstrates that the design decisions are repeatable and maintainable: sensors and control loops remain synchronized, fail-safe behaviors are triggered when needed, and the robot completes its core tasks within the available time margin. These outcomes validate the engineering methodology and provide a solid baseline for future improvements.

General Conclusion and Perspectives

Specifications review. The project aimed for complete autonomy over a 100-second match, precise navigation, and reliable interaction with game elements. All major technical objectives have been achieved.

Contributions. The main contributions are a distributed RPi5/STM32/ESP32 architecture with micro-ROS, Hybrid A* path planning in Python, and LIDAR/vision perception for obstacle avoidance and localization.

Impact. The competition performance (1st nationally, 2nd worldwide) validates these technical choices and strengthens the visibility of the team and INSAT internationally.

Perspectives for the PFA (Final Year Project). Natural next steps include the integration of full SLAM, improvement of the multi-robot strategy, and control optimization for even more dynamic environments.

Abstract

Résumé (FR). Ce rapport présente la conception et la réalisation d'un robot autonome pour Eurobot 2026, couvrant l'architecture, la mécanique, l'électronique et le logiciel. L'architecture distribuée combine un Raspberry Pi 5 (ROS2), un STM32 et trois ESP32 reliés par micro-ROS, avec une navigation basée sur une odométrie précise, un planning Hybrid A* et un évitement d'obstacles via LIDAR LD19. La mécanique s'appuie sur un châssis modulaire et un système de préhension pneumatique à ventouses, issu d'une démarche de conception multi-concepts. La perception visuelle utilise des marqueurs ArUco et un modèle de pickability embarqué (OpenVINO) pour fiabiliser la manipulation. Les validations montrent un drift de 3 mm après 100 s, et l'équipe obtient la 2ème place mondiale.

Abstract (EN). This report presents the design and realization of an autonomous robot for Eurobot 2026, spanning mechanical, electronic, and software subsystems. The distributed architecture couples a Raspberry Pi 5 (ROS2), an STM32, and three ESP32 boards through micro-ROS, with navigation relying on accurate odometry, Hybrid A* planning, and LD19 LIDAR obstacle avoidance. The mechanical solution uses a modular chassis and a pneumatic suction-based gripper chosen through a multi-concept design study. Visual perception combines ArUco markers with an embedded OpenVINO pickability model to improve grasp reliability. Validation shows a 3 mm drift after 100 s, and the team achieved 2nd place worldwide.

Bibliography and Webography

[1] ROS2 Documentation, https://docs.ros.org, accessed May 2026.

[2] micro-ROS Documentation, https://micro.ros.org, accessed May 2026.

[3] Eurobot 2026 Official Rules, official document, accessed May 2026.

[4] Pololu motor datasheet (product 4742), https://www.pololu.com/product/4742, accessed May 2026.

[5] JSUMO JS7444 wheel datasheet, https://www.jsumo.com/js7444-aluminum-silicone-wheel-pair-74mm-diameter, accessed May 2026.

[6] N20 micro DC gear motor datasheet, https://www.aslongdcmotor.com/photo/aslongdcmotor/document/26549/12MM%20Small%20Dc%20Gear%20Motor_PDF00.pdf, accessed May 2026.

[7] Omnitrack 9100 series datasheet, https://resources.omnitrack.com/files/pdf/9100_2D.pdf, accessed May 2026.

[8] Eurobot 2026 General Rules, https://www.eurobot.org/wp-content/uploads/2025/10/Eurobot2026_Rules_1.0_FR.pdf, accessed May 2026.

[9] Optical encoder datasheet, https://www.elektrosms.cz/ElektroSMS/media/com/D0000000431/15-32_37_E_2101.pdf, accessed May 2026.

[10] ROS2 Jazzy documentation, https://docs.ros.org/en/jazzy/index.html, accessed May 2026.

[11] micro-ROS wiki (Vulcanexus), https://micro.vulcanexus.org/, accessed May 2026.